Git-Driven Deployment Engine with AI Failure Summary

Development Phases & Implementation Roadmap

Project Goal: Build an intelligent DevOps automation platform that connects Git repositories with CI/CD workflows, monitors deployments, collects failure logs, and generates AI-powered failure summaries and recommendations.

Recommended approach: Build the system incrementally. First make the deployment pipeline work, then add monitoring, AI analysis, dashboard features, and finally cloud/production features.

Core System Flow

```
Git Push → CI/CD Trigger → Build & Test → Docker → Deployment → Monitoring → Failure Detection → Log Collection → AI Analysis → Failure Summary → Dashboard / Notification
```

Phase 1 — Project Setup & Architecture

• Create the project repository and initial folder structure.

• Recommended stack: React.js + Bootstrap, Flask, PostgreSQL, GitHub Actions, Docker.

• Keep the architecture modular so Git integration, deployment, logs, AI, database, and dashboard can evolve independently.

Phase 2 — GitHub Integration

• Start with GitHub rather than implementing multiple Git/CI platforms at once.

• Connect the system to a repository and detect push events.

• Capture repository, branch, commit ID, author, and event information.

• Use GitHub Webhooks and/or GitHub API as the integration layer.

Phase 3 — CI/CD Pipeline

• Create the first working automated pipeline using GitHub Actions.

• Pipeline sequence: checkout code → install dependencies → run tests → build Docker image → deploy.

• Use a simple Flask demo application as the first deployment target.

• Do not add advanced infrastructure until this basic pipeline works reliably.

Phase 4 — Docker Integration

• Create a Dockerfile for the application.

• Build a Docker image and run the application inside a container.

• Integrate Docker build/run steps into the CI/CD workflow.

• Keep Kubernetes and multi-cloud deployment for later stages.

Phase 5 — Deployment Monitoring

• Track deployment status in real time or near real time.

• Record build, test, Docker, and deployment status.

• Maintain deployment history including repository, branch, commit, timestamps, and status.

• Represent both successful and failed deployments clearly.

Phase 6 — Database

• Connect PostgreSQL to the backend.

• Recommended initial entities: users, repositories, deployments, deployment_logs, failure_analysis.

• Store deployment history, logs, failure analysis, summaries, and recommendations.

• Design relationships so each deployment can be linked to its logs and AI analysis.

Phase 7 — Log Collection Engine

• Collect build logs, test logs, Docker logs, stack traces, and error messages.

• Extract the relevant failure information from large logs.

• Store historical logs for later analysis and troubleshooting.

• Make the log-processing layer independent from the AI layer.

Phase 8 — AI Failure Analysis

• Build the core AI analysis pipeline.

• Flow: raw log → preprocessing → error extraction → classification/analysis → probable root cause → recommendation.

• Start with a practical log-analysis approach rather than attempting to train a large model immediately.

• Test the analyzer using common dependency, syntax, Docker, configuration, and runtime failures.

Phase 9 — Failure Summary Generator

• Convert the AI analysis into a concise developer-friendly result.

• Recommended output: failure status, failure type, key error, probable root cause, why it happened, recommended action, and severity.

• Keep the summary short and actionable while allowing access to the complete logs.

Phase 10 — React Dashboard

• Create a centralized dashboard for deployments and failures.

• Show repositories, recent deployments, success/failure counts, deployment history, and AI summaries.

• Provide a detailed deployment page containing pipeline status, error information, AI analysis, and full logs.

• Use charts/visual reports where useful for system health and failure trends.

Phase 11 — Notifications

• Add deployment success/failure notifications after the core system is stable.

• Start with one notification channel such as email.

• Include the repository, commit, failure reason, and AI-generated summary in failure alerts.

• Additional integrations such as Slack and Microsoft Teams can be added later.

Phase 12 — AWS Deployment

• Move the working application to AWS only after the local version is stable.

• Use EC2 for application/deployment infrastructure.

• Use S3 for suitable storage requirements and CloudWatch for monitoring/logging where appropriate.

• Validate security, networking, environment variables, and deployment reliability in the cloud.

Phase 13 — Security & Admin Panel

• Add authentication and role-based access control.

• Admin capabilities can include users, repositories, CI/CD settings, deployment parameters, and system activity.

• Developer access can focus on assigned repositories, deployments, logs, and AI summaries.

• Protect GitHub credentials, API keys, database credentials, and deployment secrets.

Phase 14 — Testing

• Create intentional failures to validate the complete failure-analysis workflow.

• Test dependency failures, syntax errors, Docker failures, configuration errors, and runtime failures.

• Verify that logs are captured, failures are classified, summaries are meaningful, and recommendations are useful.

• Test both successful and failed deployment paths.

Phase 15 — Final Cloud Deployment & Documentation

• Complete the production-style architecture and final deployment.

• Prepare architecture and ER diagrams, API documentation, screenshots, test cases, performance results, and README.

• Document setup instructions, environment variables, CI/CD workflow, AI analysis flow, and known limitations.

• Prepare a clear project demonstration showing a successful deployment and an intentional failure followed by AI analysis.

Five Major Development Milestones

Milestone 1 — Basic Deployment Engine

```
GitHub → GitHub Actions → Build → Test → Docker → Deploy
```

Milestone 2 — Monitoring

```
GitHub → CI/CD → Deployment Engine → Status + Logs + Database
```

Milestone 3 — AI Failure Analysis

```
Failed Deployment → Logs → Log Processor → AI Analyzer → Root Cause → Summary →
Recommendation
```

Milestone 4 — Dashboard

```
React Dashboard → Flask REST API → PostgreSQL → Deployments / Logs / AI Analysis
```

Milestone 5 — Cloud & Production

```
AWS EC2 + S3 + CloudWatch + Authentication + Notifications + Security
```

Recommended First Implementation

1. Create the GitHub repository.

2. Create a simple Flask application.

3. Dockerize the Flask application.

4. Create the GitHub Actions workflow.

5. Automatically build and test on every Git push.

6. Create an intentional deployment failure.

7. Capture and store the failure logs.

8. Build the AI failure analyzer.

9. Connect PostgreSQL for deployment and analysis history.

10. Build the React dashboard.

Important Development Rule

Do not build everything at once. First make one complete pipeline work: **Git push** → **build** → **test** → **Docker** → **deployment** → **success/failure.** Once this is reliable, add log collection, AI analysis, database, dashboard, notifications, and cloud deployment in that order.

Reference: Project synopsis — Git-Driven Deployment Engine with AI Failure Summary. This roadmap follows the modules, objectives, technology stack, scope, and expected outcomes described in the synopsis.